

Übungsblatt 11

Vorlesung „Cybersicherheit“

Sommersemester 2024

Übung Software-Exploits

Im Laufe der Übung werden wir verschiedene Tools zur Analyse von Programmen verwenden. Wir empfehlen die Verwendung einer virtuellen Maschine (z.B., VirtualBox¹) mit Windows oder Linux, um Probleme mit bestehender Software zu vermeiden. Alle Aufgaben können jedoch auch ohne eine virtuelle Maschine erledigt werden.

Secuso-VM Zur Lösung der Aufgaben stellen wir eine virtuelle Linux-Maschine (VM) zur Verfügung. Um die VM zu verwenden, installieren Sie Oracle VirtualBox und importieren Sie die VM. Der Benutzername und das Passwort in der VM lauten beide `user`. Alle Werkzeuge, die Sie für diesen Lehrgang benötigen, sind in der VM installiert. Die VM kann über den folgenden Link heruntergeladen werden:

<https://uni-duisburg-essen.sciebo.de/s/WR5MKiH97UntQkh>

Einführung in die x86 Programmierung

In dieser Aufgabe werden Sie ein „Hello World“ Assembler Programm analysieren. Das `hello-x86` Programm in der Intel 32-Bit ISA (für Linux i386) geschrieben. Gehen Sie in den in der ZIP-Datei enthaltenen Ordner `1_hello`. Um das Programm `hello-x86` im Debugger `gdb` zu starten gehen Sie wie folgt vor:

- Laden Sie das Programm in `gdb`:
`$ gdb ./hello-x86`
- Um das Programm zu disassemblieren können Sie folgendes Kommando in `gdb` verwenden:
`disassemble _start`
Alternativ dazu können Sie auch *Cutter* verwenden.
- Starten Sie das Programm in `gdb`:
`start`
und „steppen“ Sie Instruktion für Instruktion durch das Programm:
`stepi` (oder kurz: `si`) (*step instruction*)
- Verwenden Sie `kill` und `start` um das Programm im Debugger zu terminieren und neu zu starten.

Siehe Appendix A für mehr `gdb` Befehle. Machen Sie sich Notizen (z.B. in einer Assembler Datei (`.S`) oder mittels der Kommentarfunktion von *Cutter*) zu dieser Aufgabe.

¹<https://www.virtualbox.org/>

Aufgabe 1.

- a) Disassemblieren Sie die Routine `_start` in `hello-x86` und erklären Sie kurz jede Instruktion.
- b) Debuggen Sie das Programm `hello-x86` mit dem Debugger `gdb` und bestimmen Sie die tatsächlichen Werte und Adressen in den Registern jeweils bevor die `int 0x80` Instruktion ausgeführt wird. Erklären Sie die Bedeutung der verschiedenen Register.

Hinweis: Appendix B beinhaltet einen Ausschnitt aus der „System Call Table“.

- c) Wie funktionieren Linux System-Calls auf x86 32-Bit? Wie werden Parameter übergeben? Wie wird der Rückgabewert übergeben?

Links: Nützliche Links zum Thema Assembler Programmierung:

- <http://www.cs.virginia.edu/~evans/cs216/guides/x86.html>
- <http://unixwiz.net/techtips/x86-jumps.html>
- <http://www.felixcloutier.com/x86/>

Data-Only Attack

Der Codeausschnitt in Listing 1 beinhaltet eine fehlerhafte Passwort-Authentifizierung. Diese Schwachstelle ermöglicht eine erfolgreiche Authentifizierung trotz inkorrektem Passwort. Das Ziel dieser Übung ist es diese Schwachstelle zu analysieren, nachzuvollziehen, und auszunutzen.

Mehr Informationen zu den benutzten libc-Funktionen finden Sie auch in dem Man-pages:

- `$ man 3 strcpy` (unter Linux) oder unter <https://linux.die.net/man/3/strcpy>
- `$ man 3 strcmp` oder <https://linux.die.net/man/3/strcmp>

Aufgabe 2.

1. Beschreibe den Fehler in Listing 1. Was sind die Auswirkungen des Fehlers? Wie könnte dieser Fehler behoben werden?
2. Skizzieren Sie das Stackframe-Layout der Funktion `pw_check`.
3. Finde eine gültige Eingabe, die *nicht* „cyssec“ ist, damit das Programm die Nachricht „Gut gemacht!“ ausgibt. Warum funktioniert Ihr Angriff?
4. Was würde passieren, wenn der Compiler die Reihenfolge der lokalen Variablen ändert? Erläutern Sie kurz.

```

1 int pw_check(char *password) {
2     int auth = 0;
3
4     // Allokiere Speicher auf dem Stack
5     char pw_buffer[16] = {0};
6
7     // Kopiere den *gesamten* String "password" nach "pw_buffer"
8     strcpy(pw_buffer, password);
9
10    // Falls das eingegeben Passwort "cyssec" ist,
11    // setze die Variable "auth"
12    if (strcmp(pw_buffer, "cyssec") == 0)
13        auth = 1;
14
15    return auth;
16 }
17
18 int main(int argc, char *argv[]) {
19
20     if (argc != 2) {
21         printf("Benutzung: %s <password>\n", argv[0]);
22         exit(0);
23     }
24
25     // Das eingegebene Passwort wird in der Variable "pw" gespeichert.
26     char *pw = argv[1];
27
28     // Die Funktion "pw_check" prüft das Passwort.
29     if (pw_check(pw)) printf("Gut gemacht!\n");
30     else printf("Zzzzzz...\n");
31 }

```

Listing 1: Passwort-Authentifizierung

Bonusaufgabe

Der Entwickler hat nun das Problem in Listing 1 erkannt und hat die anfällige Passwort-Authentifizierung korrigiert. Analysieren Sie nun das Programm `canary` mit der aktualisierten Authentifizierungsroutine.

Aufgabe 3.

1. Was hat sich in der `pw_check` Funktion verändert?
2. Wie hat der Entwickler den Fehler korrigiert? Beschreiben Sie es mithilfe des Stackframe-Layouts der `pw_check` Funktion.
3. Beschreiben Sie wie diese Korrektur umgangen werden kann.
4. Übermitteln Sie erneut eine Eingabe die nicht `cyssec` ist, welche die Nachricht „Gut gemacht!“ ausgibt.

A GNU Debugger gdb

The following gdb commands may help you to solve this exercise:

- Use Intel Assembly Syntax: `set disassembly-flavor intel`
- `quit` / `q`: Exit gdb
- `list`: Show lines of the source code
- `disassemble function`: Disassemble a function
 - `disassemble main`: Disassemble the main function
- `break` / `b`: Set a breakpoint
 - `break *0x4009c0`: Set a breakpoint at address 0x4009c0
 - `break main`: Set a breakpoint at the main function
- `run [arglist]`: Run the program with arguments *arglist*
- `step` / `s`: Execute one source line. Steps into functions calls.
- `next` / `n`: Execute one source line. Steps over function calls.
- `step instruction` / `si`: Execute one assembler instruction. Steps into function calls.
- `next instruction` / `ni`: Execute one assembler instruction. Steps over function calls.
- `continue`: Continue program execution until the next breakpoint is hit
- `info registers` / `i r`: Print the processor register values.
- `print [expression]`: Show value of *expression*
 - `print /x $rax`: Print the value of the %rax register in hexadecimal (x) notation
- `x/[Nuf] expression`: Examine memory at address *expression*, whereas N indicates the Number of units to display, u the unit size, and f the printing format.
 - `x/4wx $rsp`: Print the top four (4) 32 bit words (w) on the stack (\$rsp) in hexadecimal (x) notation
 - `x/4gx $rsp`: Print the top four (4) 64 bit words (g) on the stack (\$rsp) in hexadecimal (x) notation
 - `x/1s $rsi`: Display one String (1s) starting from the address stored in (\$rsi).

The following commands are provided by gdb extensions, such as `pwndbg` or `gef`. `pwndbg` is loaded by default on the lab virtual machine.

- `context`: print all kinds of information (stack, registers, source if available)
- `telescope $eax 40`: Dereference the register `eax` and print 40 words of data, dereferencing pointers.
- `registers`: pretty-print register contents
- `stack`: pretty-print stack contents

B Linux System Calls für x86 32-bit

Tabelle 1 zeigt einen vereinfachten Ausschnitt aus der “System Call Table” des Linux Kernels. Darin sollten sich alle wichtigen System Calls befinden, die Ihnen während der Übungen begegnen.

Tabelle 1: First 14 Linux Systems Calls on x86 32-bit

Number	Name	eax	ebx	ecx	edx	esi	edi
0	sys_restart_syscall	0x00	-	-	-	-	-
1	sys_exit	0x01	int error_code	-	-	-	-
2	sys_fork	0x02	struct pt_regs *	-	-	-	-
3	sys_read	0x03	unsigned int fd	char *buf	size_t count	-	-
4	sys_write	0x04	unsigned int fd	char *buf	size_t count	-	-
5	sys_open	0x05	char *filename	int flags	int mode	-	-
6	sys_close	0x06	unsigned int fd	-	-	-	-
7	sys_waitpid	0x07	pid_t pid	int *stat_addr	int options	-	-
8	sys_creat	0x08	char *pathname	int mode	-	-	-
9	sys_link	0x09	char *oldname	char *newname	-	-	-
10	sys_unlink	0x0a	char *pathname	-	-	-	-
11	sys_execve	0x0b	char* filename	char** argv	char** envp	struct pt_regs *	-
12	sys_chdir	0x0c	char *filename	-	-	-	-
13	sys_time	0x0d	time_t *tloc	-	-	-	-
14	sys_mknod	0x0e	char *filename	int mode	unsigned dev	-	-
			...				

C ASCII Table

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Abbildung 1: ASCII Tabelle